Ứng dụng Notes API: Python, HTTP và SQL

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Scripting, ứng dụng và database

Script là chương trình nhỏ tự động hóa công việc. Một script tốt có input rõ, kiểm tra lỗi, mã thoát (exit code) đáng tin, không in bí mật và có thể chạy lại mà không gây hại. Bắt đầu bằng bài toán cụ thể: kiểm tra một URL có phản hồi hay không; ghi thời điểm và status; trả mã 0 khi đúng, khác 0 khi lỗi. Trước khi dùng script trong CI, hãy tự chạy và kiểm tra trường hợp thành công lẫn thất bại.

Ứng dụng thường nhận cấu hình qua biến môi trường hoặc file cấu hình, kết nối tới database và ghi log. Cùng một mã nguồn có thể chạy ở local/test/production với cấu hình khác nhau. Không đặt mật khẩu trực tiếp trong mã hoặc commit file **.env** chứa bí mật. Health endpoint chỉ nên nói ứng dụng có thể đáp ứng điều gì; kiểm tra quá nặng có thể làm tình trạng xấu hơn.

Database quan hệ lưu dữ liệu trong bảng; mỗi hàng là một bản ghi, mỗi cột là thuộc tính. SQL **SELECT** đọc, **INSERT** thêm, **UPDATE** sửa, **DELETE** xóa. Một giao dịch (transaction) nhóm nhiều thao tác để giữ dữ liệu nhất quán theo quy tắc database. Backup chỉ có giá trị khi đã thử restore. Khi triển khai phiên bản mới, thay đổi schema có thể không tương thích với phiên bản cũ, nên cần nghĩ đến rollback trước khi migration.

Bài tập thiết kế ứng dụng mẫu

Ứng dụng xuyên suốt sẽ là API ghi chú đơn giản: tạo ghi chú, liệt kê và kiểm tra sức khỏe. Vẽ bốn thành phần: client, API, database, nơi chứa log. Ghi data nào cần tồn tại sau restart và data nào không. Hãy tự chọn Python, Node.js hoặc ngôn ngữ bạn muốn học; mục tiêu DevOps là hiểu đủ mã để chạy và kiểm tra, không cần xây sản phẩm lớn ngay.

Thực hành ứng dụng mẫu từ đầu đến cuối

Để các bài sau có một đối tượng chung, tài liệu kèm một Notes API nhỏ ở **lesson/labs/notes-api/app.py**. Nó dùng thư viện chuẩn Python và SQLite; không cần cài framework web hay database server. Đây là ứng dụng học tập, không phải mẫu triển khai production. Khi đọc code, đừng cố nhớ từng dòng: trước tiên tìm nơi mở cổng, nơi nhận request và nơi lưu dữ liệu.

Chuẩn bị và chạy

Điều kiện: Python 3 đã được cài, bạn có quyền tạo file trong thư mục lab. Mở terminal ở **lesson/labs/notes-api**. Chạy **python --version** hoặc **py --version** trên Windows để xác nhận trình thông dịch. Nếu báo không nhận ra lệnh, dừng lab và cài Python từ nguồn chính thức hoặc nhờ Codex hướng dẫn theo hệ điều hành của bạn. Khi Python đã sẵn sàng, chạy **python app.py** (Windows có thể là **py app.py**). Terminal sẽ in địa chỉ **http://127.0.0.1:8000**. Giữ cửa sổ này mở.

Ở terminal thứ hai, chạy **curl http://127.0.0.1:8000/health** trên Linux/macOS. PowerShell có thể dùng **Invoke-RestMethod http://127.0.0.1:8000/health**. Kết quả mong đợi có **status: ok** và **version: lab-1**. Nếu kết nối thất bại, kiểm tra terminal thứ nhất còn chạy không, cổng 8000 có bị ứng dụng khác chiếm không, và bạn có gõ đúng **127.0.0.1** không. Không thay nhiều thứ cùng lúc.

Trên PowerShell, thử tạo ghi chú bằng **Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/notes -ContentType 'application/json' -Body '{"body":"Ghi chu dau tien"}'**. Trên Linux/macOS, dùng **curl -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d '{"body":"Ghi chu dau tien"}'**. Kết quả mong đợi là HTTP 201 với **id** và nội dung. Gọi **GET /notes** để thấy bản ghi. Thử gửi **{}** và dự đoán HTTP status trước khi xem: ứng dụng trả 400 vì thiếu **body**.

Dừng ứng dụng bằng Ctrl+C rồi chạy lại. Gọi **GET /notes**; ghi chú vẫn còn vì SQLite ghi vào **notes.db**. Tạm đổi tên file này trong lab rồi chạy lại; danh sách trở nên rỗng vì ứng dụng mở database mới. Khôi phục tên file cũ để lấy lại dữ liệu. Thí nghiệm này giúp bạn phân biệt dữ liệu nằm trong bộ nhớ tiến trình với dữ liệu lưu bền trên đĩa. Đừng đổi tên file database của ứng dụng thật khi chưa có backup.

Đọc một request xuyên các lớp

Khi gọi **GET /notes**, chương trình client tạo HTTP request tới địa chỉ loopback **127.0.0.1** và port 8000. Hệ điều hành chuyển kết nối đến tiến trình Python đang lắng nghe. Handler đọc path **/notes**, mở SQLite, chạy câu **SELECT**, chuyển các hàng thành JSON rồi gửi HTTP response. Hãy vẽ chuỗi này lên giấy và đánh dấu: IP, port, process, file database, SQL, JSON và status code. Sau đó giải thích vì sao **GET /unknown** trả 404 dù server đang chạy tốt.

Log in ở terminal thứ nhất cho biết request và status. So sánh log khi gọi **/health**, **/notes** và **/unknown**. Nếu terminal không hiện request, lỗi có thể xảy ra trước khi request tới tiến trình: client gọi sai host/port hoặc server không chạy. Nếu log hiện 500, request đã tới ứng dụng và bạn cần xem exception. Đây là cách đầu tiên dùng log để phân lớp lỗi.

Bài tập xử lý lỗi có hướng dẫn

Tình huống A: ứng dụng báo cổng đã được sử dụng. Trước hết xác nhận một bản **app.py** khác có đang chạy không. Nếu có, dừng bản cũ bằng Ctrl+C. Nếu không, có thể chạy lab ở cổng khác bằng cách đặt biến **NOTES_PORT** trong môi trường rồi khởi động lại. Đừng tắt một tiến trình lạ chỉ vì nó dùng cùng cổng.

Tình huống B: **GET /health** hoạt động nhưng **POST /notes** trả 400. Điều này cho biết mạng và process cơ bản hoạt động. Với ứng dụng lab hiện tại, kiểm tra JSON body, key **body** và giới hạn nội dung; code chưa kiểm tra header **Content-Type** dù client nên gửi **application/json**. Đừng sửa DNS hoặc firewall khi request đã được API trả 400.

Tình huống C: ghi chú mất sau khi xóa thư mục lab. Ứng dụng lưu vào **notes.db** trong thư mục hiện tại theo mặc định. Đây là lý do lệnh được chạy từ đâu quan trọng. Bạn có thể chỉ định đường dẫn rõ qua biến **NOTES_DB**, nhưng hãy ghi lại vị trí và sao lưu trước khi xóa môi trường.

Kiểm tra hiểu sâu

- Nếu gọi **127.0.0.1:8001** trong khi ứng dụng nghe 8000, lớp nào sai?
- Nếu **/health** trả 200 nhưng **/notes** lỗi do database, health endpoint đã chứng minh được điều gì và chưa chứng minh được điều gì?
- Vì sao thao tác thêm ghi chú dùng POST còn liệt kê dùng GET?
- Vì sao truy vấn SQL trong code dùng tham số **?** thay vì ghép trực tiếp chuỗi người dùng? Gợi ý: tránh SQL injection và vấn đề escaping.

Đáp án định hướng: (1) cổng của kết nối; (2) tiến trình có thể trả lời health, chưa chứng minh đường database; (3) GET đọc, POST tạo tài nguyên theo API đơn giản này; (4) câu lệnh tham số tách dữ liệu khỏi cấu trúc SQL. Hãy trả lời bằng lời của mình trước khi xem gợi ý.

Đọc script và truy vấn SQL như người vận hành

Một script kiểm tra dịch vụ cần trả exit code đáng tin. Trong nhiều shell, 0 biểu thị thành công và số khác 0 biểu thị lỗi. CI dựa vào mã thoát để quyết định job xanh hay đỏ; in chữ "ERROR" rồi trả 0 có thể khiến pipeline tưởng mọi thứ ổn. Script cũng cần timeout: nếu request treo mãi, pipeline và công việc vận hành bị kẹt. Khi có retry, nên giới hạn số lần và khoảng chờ; retry vô hạn có thể che sự cố.

Code Notes API dùng câu **SELECT id, body FROM notes ORDER BY id** để đọc; **INSERT INTO notes (body) VALUES (?)** để thêm. Dấu **?** là tham số của SQLite, giúp tách dữ liệu người dùng khỏi cú pháp SQL. Nếu ghép trực tiếp **body** vào chuỗi truy vấn, một input có dấu nháy có thể làm hỏng cú pháp và input độc hại có thể thay đổi ý nghĩa truy vấn. Khi đọc code ứng dụng, hãy xác định query nào đọc, query nào ghi, file dữ liệu ở đâu và lỗi database được log thế nào.

Một transaction có ý nghĩa khi nhiều thay đổi phải nhất quán. Ví dụ tạo đơn hàng gồm dòng hàng và cập nhật số lượng tồn: nếu dòng hàng được ghi nhưng cập nhật tồn thất bại, database có thể rơi vào trạng thái sai nếu không quản lý giao dịch. Notes API chỉ thêm một hàng nên ví dụ đơn giản. Ở giai đoạn sau, bạn sẽ thấy tại sao migration database cần tính tới phiên bản ứng dụng cũ và mới.

Bài tập mở rộng app có kiểm soát

Thêm endpoint **GET /version** trả version hiện tại từ biến môi trường. Trước khi sửa, viết kết quả mong đợi; sau khi sửa, gọi endpoint và kiểm tra status/body. Sau đó cố ý không đặt biến môi trường để xem default. Đây là bài tập tự làm, không cần đưa ngay lên cloud. Nếu bạn chưa tự tin sửa Python, hãy nhờ Codex hướng dẫn từng dòng và yêu cầu nó giải thích thay đổi trước khi áp dụng.

Chạy và đọc test thật của Notes API

File **lesson/labs/notes-api/test_app.py** dùng thư viện **unittest**. **setUpClass** tạo thư mục database tạm và khởi động server trên cổng tự chọn; **tearDownClass** dừng server và dọn dữ liệu thử. Test health kiểm tra HTTP 200 cùng JSON **status: ok**; test tạo ghi chú gửi POST rồi GET để xác minh dữ liệu đã lưu; test input rỗng xác nhận HTTP 400. Đây là integration test vì nó thực sự đi qua HTTP handler và SQLite, không chỉ gọi một hàm riêng lẻ.

Mở terminal ở thư mục **lesson/labs/notes-api** và chạy **python -m unittest -v test_app.py**. Mong đợi ba test **ok** và tổng kết **OK**. Nếu một test fail, đọc tên test, dòng assertion và phản hồi thực tế. Đừng chỉ sửa giá trị kỳ vọng cho xanh: hãy hỏi hành vi mong muốn là gì. Thử cố ý đổi **status: ok** trong code thành **status: bad**, chạy lại và dự đoán test nào đỏ. Hoàn nguyên code sau thí nghiệm.

Test có thể pass trên máy nhưng fail CI vì khác Python, working directory, quyền file hoặc timing. File test dùng cổng do hệ điều hành chọn để tránh xung đột với app chạy trên 8000. Nó cũng dùng database tạm để không làm bẩn ghi chú cá nhân. Hai quyết định này làm test lặp được hơn. Nếu test thỉnh thoảng đỏ, ghi tần suất và điều kiện; test không ổn định không nên được giải quyết bằng cách bỏ qua lỗi mà không tìm nguyên nhân.

Đọc test như hợp đồng hành vi

Hãy viết bảng ba cột: input, expected status, expected data. Dòng 1: **GET /health**, 200, **status=ok**. Dòng 2: POST JSON có body hợp lệ, 201, trả ID. Dòng 3: POST JSON **{}**, 400, báo invalid_note. Sau đó bổ sung một trường hợp bạn muốn test, ví dụ GET path không tồn tại phải 404. Nếu viết được test mới rồi thấy nó fail trước khi sửa code, bạn đã dùng test để làm rõ yêu cầu.

Đọc API như một hệ thống vận hành, không chỉ là mã Python

Một request **POST /notes** đi qua TCP/HTTP, router, validation, logic nghiệp vụ, truy vấn DB, commit transaction rồi trả response. Ở mỗi bước có dạng lỗi khác nhau: request sai trả 4xx, code exception thường 5xx, DB bận có thể timeout, response mất sau commit tạo tình huống client không biết việc ghi đã thành công. Người DevOps cần đọc code đủ để biết endpoint health kiểm tra gì, log có request ID không, DB file nằm đâu, biến môi trường nào bắt buộc và migration chạy lúc nào. Không cần viết toàn bộ ứng dụng như backend engineer, nhưng không thể vận hành một "hộp đen" hoàn toàn.

HTTP contract và idempotency

GET dùng để đọc; POST thường tạo; PUT thường thay một tài nguyên theo ID; PATCH sửa một phần; DELETE xóa. Method chỉ là convention nếu server viết sai, nên đọc contract/test thực. Status 200/201/204 cho những kiểu thành công khác nhau; 400/422 cho input sai theo thiết kế; 401 khác 403; 404 có thể do route/ID; 500 nghĩa server lỗi, không tự xác định DB hay code. Khi client retry POST vì timeout, server có thể đã commit lần đầu. Với hành động tài chính/đơn hàng, dùng idempotency key hoặc thiết kế unique constraint + transaction; retry mù có thể tạo bản ghi đôi.

API nên log request ID, method, route, status, latency và lỗi đã được phân loại; tránh log secret, token hoặc toàn bộ dữ liệu người dùng. Metrics nên có số request, tỷ lệ lỗi và histogram latency; đừng dùng user ID làm label Prometheus vì cardinality nổ. Trace ID nối request qua proxy, API, queue và DB nếu stack hỗ trợ. Một endpoint **/health** có thể chỉ chứng minh process còn sống; readiness và smoke test phải phù hợp nghiệp vụ.

SQL và transaction qua một ví dụ

Giả sử tạo note và một bản ghi audit cùng lúc. Nếu chèn note thành công nhưng chèn audit thất bại, bạn muốn cả hai cùng rollback hay chấp nhận audit bất đồng bộ? Transaction **BEGIN -> INSERT note -> INSERT audit -> COMMIT** cho tính nguyên tử trong một DB. Nếu app đóng connection trước commit hoặc exception xảy ra, thay đổi có thể rollback tùy cách quản lý transaction. Với Python **sqlite3** phiên bản hiện hành, tài liệu khuyến nghị điều khiển qua **Connection.autocommit**; code cũ dùng **isolation_level** có hành vi legacy. Khi đọc app mẫu, không suy đoán từ cú pháp SQL; xem connection mode, commit/rollback và test.

Parameterized query tách dữ liệu người dùng khỏi cú pháp SQL: **cur.execute("SELECT ... WHERE id = ?", (note_id,))**. Ghép chuỗi trực tiếp vào SQL mở đường injection và lỗi escaping. Unique constraint/index là lớp bảo vệ dữ liệu; validation ở API một mình không chống race condition giữa hai request. Index tăng tốc truy vấn phù hợp nhưng làm ghi tốn thêm và chiếm đĩa; xem query plan trước khi thêm hàng loạt index.

Lab: theo một request đến file DB

Điều kiện: Python tương thích với app mẫu; dùng thư mục **lesson/labs/notes-api** và venv riêng, không dùng DB production. Đọc README/app.py trước để xác định lệnh chạy và biến môi trường. Chạy test có sẵn của dự án nếu bạn muốn kiểm chứng logic; ở đây bài học tập trung quan sát. Khởi động API trong terminal A với DB tạm và gọi request tạo note ở terminal B theo endpoint trong mã nguồn. **Kết quả dự kiến:** response có ID/trạng thái theo code; DB file xuất hiện ở đường cấu hình; query SELECT thấy bản ghi. Đừng ghi output mẫu như thể đã chạy trên máy bạn: port, ID và nội dung phụ thuộc môi trường.

Gây lỗi an toàn: đổi DB path sang thư mục không tồn tại hoặc không có quyền trong **bản cấu hình lab**, rồi gọi lại. Ghi HTTP status, exception log, file DB có được tạo không. Sau đó phục hồi path và xác nhận chức năng. Lỗi DB path khác lỗi validation request; người trực ca có thể phân biệt nhờ log, timing và phép thử đọc/ghi. Dọn file DB tạm sau khi xác nhận đúng đường dẫn. Nếu app dùng SQLite và nhiều process/replica, tìm hiểu giới hạn concurrency/locking trước khi kết luận mô hình scale được trên Kubernetes.

Migration và release tương thích

Thêm cột **priority** có thể làm theo hướng expand/contract: thêm cột có default/nullable để app cũ vẫn chạy; deploy app mới bắt đầu ghi/đọc; backfill dữ liệu cũ theo batch có quan sát; khi mọi phiên bản cũ đã rời hệ thống mới bắt buộc cột/xóa đường tương thích ở release sau. Nếu app mới yêu cầu cột trước khi migration chạy, rollout có thể tạo 500 dù Pod Ready. Nếu rollback app sau khi dữ liệu đã viết theo schema mới, cần biết code cũ có đọc được không. Backup trước migration và restore drill quan trọng hơn một file SQL “DOWN” chưa từng chạy thử.

Câu hỏi có lời giải

API trả 500 sau deploy, CPU thấp. Xem request ID/log/trace, DB connection/path/schema/quyền, dependency và release diff; CPU thấp không loại trừ I/O wait hoặc lock. **Cùng request gửi hai lần tạo hai note.** Đây là vấn đề idempotency; cần key/constraint/transaction thay vì chỉ giảm timeout. **Pod health xanh nhưng POST /notes lỗi.** Health check có thể chỉ kiểm tra process; thêm smoke test nghiệp vụ và metric theo route. **Một DB SQLite file đặt trong writable layer container.** Khi container/Pod bị thay, dữ liệu có thể mất; phải thiết kế storage/DB riêng và backup, không chỉ tăng replica.

Nguồn: Python sqlite3 transaction control, SQLite documentation, HTTP semantics RFC 9110.

Đọc lỗi ứng dụng như một người vận hành

Giả sử **POST /notes** trả 500 trong 2% request. Đầu tiên tìm route, thời gian, revision và request ID; xem lỗi có tập trung ở một Pod/user/input hay mọi request. Nếu log nói **sqlite3.OperationalError: database is locked**, không kết luận ngay “SQLite hỏng”: kiểm tra transaction dài, số process/replica, timeout, path/mount và recent load. Nếu lỗi **no such table**, xem migration/schema version và DB path: app có thể vô tình tạo file DB mới trống do đường dẫn sai. Nếu lỗi **permission denied**, kiểm tra UID/GID/dir traverse và mount read-only. Cùng HTTP 500 nhưng ba nguyên nhân/đường khôi phục khác nhau.

Contract test và test tầng nào?

Unit test logic validation không cần DB; integration test tạo DB tạm, chạy migration và endpoint, kiểm tra transaction; end-to-end test gọi đường ingress/Service như client; smoke test production chọn vài chức năng cốt lõi ít tác động. Không dùng một loại test để thay tất cả. Nếu smoke chỉ gọi **/health**, một migration lỗi có thể lọt; nếu smoke tạo note, cần tài khoản/dữ liệu lab riêng và cleanup để không làm bẩn production. Test cần đo reliability của hệ thống test: flaky test bị bỏ qua sẽ mất giá trị gate.

Bài tập tính connection pool

API có 4 replica, mỗi replica pool tối đa 20 connection: tối đa 80 connection từ API, chưa tính admin/job/monitor. DB max connection 100, còn khoảng 20 cho nguồn khác trong trường hợp lý tưởng. Nếu HPA scale lên 8 replica, pool tối đa 160 có thể vượt DB trước khi CPU API cao. Thiết kế pool theo capacity DB và request concurrency, xem connection acquisition wait, idle time và timeout; thêm Pod không luôn tăng throughput. Khi phỏng vấn, nói rõ con số là giới hạn cấu hình, không phải luôn mở đủ.

Câu hỏi: **POST** timeout nhưng DB đã ghi, client retry tạo trùng; sửa ở đâu? Hợp đồng API/idempotency key, constraint và transaction, cộng timeout/retry budget; proxy chỉ là một lớp. **Câu hỏi:** health xanh nhưng DB schema sai? Health endpoint quá nông; thêm readiness/smoke theo chức năng và test migration. Nguồn: Python sqlite3, SQLite transaction docs.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://docs.python.org/3/
- https://www.sqlite.org/docs.html

Phụ lục: mã lab để đọc offline

lesson/labs/notes-api/app.py

```
"""Tiny standard-library API used by the DevOps lessons. Local lab data only."""

import json
import os
import sqlite3
from contextlib import contextmanager
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from urllib.parse import urlparse


DB_PATH = Path(os.getenv("NOTES_DB", "notes.db"))
HOST = os.getenv("NOTES_HOST", "127.0.0.1")
PORT = int(os.getenv("NOTES_PORT", "8000"))
VERSION = os.getenv("NOTES_VERSION", "lab-1")


@contextmanager
def database():
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    connection = sqlite3.connect(DB_PATH)
    try:
        connection.execute(
            "CREATE TABLE IF NOT EXISTS notes (id INTEGER PRIMARY KEY, body TEXT NOT NULL)"
        )
        connection.commit()
        yield connection
        connection.commit()
    finally:
        connection.close()


class Handler(BaseHTTPRequestHandler):
    def reply(self, status, payload):
        body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", "application/json; charset=utf-8")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        path = urlparse(self.path).path
        if path == "/health":
            self.reply(200, {"status": "ok", "version": VERSION})
        elif path == "/notes":
            with database() as db:
                rows = db.execute("SELECT id, body FROM notes ORDER BY id").fetchall()
```

```
            self.reply(200, {"notes": [{"id": row[0], "body": row[1]} for row in rows]})
        else:
            self.reply(404, {"error": "not_found"})

    def do_POST(self):
        if urlparse(self.path).path != "/notes":
            self.reply(404, {"error": "not_found"})
            return
        try:
            length = int(self.headers.get("Content-Length", "0"))
            if not 0 < length <= 4096:
                raise ValueError("body size")
            payload = json.loads(self.rfile.read(length))
            body = payload["body"]
            if not isinstance(body, str) or not body.strip() or len(body) > 1000:
                raise ValueError("body value")
        except (ValueError, KeyError, TypeError, json.JSONDecodeError):
            self.reply(400, {"error": "invalid_note"})
            return
        with database() as db:
            cursor = db.execute("INSERT INTO notes (body) VALUES (?)", (body.strip(),))
            note_id = cursor.lastrowid
        self.reply(201, {"id": note_id, "body": body.strip()})


if __name__ == "__main__":
    with database():
        pass
    server = ThreadingHTTPServer((HOST, PORT), Handler)
    print(f"Notes API {VERSION} listening on http://{HOST}:{PORT}", flush=True)
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        pass
    finally:
        server.server_close()
```